

Guía para el uso de Git y el trabajo en equipo

índice

- [Introducción](#)
- [1. Repositorio y Proyecto](#)
 - [1.1 Clonación del repositorio](#)
 - [1.2 Inicialización del proyecto en Unity](#)
 - [1.3 Configuración de Unity](#)
- [2. Prefijos](#)
 - [2.1 Acciones](#)
 - [2.2 Áreas](#)
 - [2.3 Uso de Prefijos](#)
- [3. Flujo de trabajo](#)
 - [3.1 Configuración inicial](#)
 - [3.2 flujo común al hacer cambios](#)
 - [3.2.1 Antes de cualquier cambio](#)
 - [3.2.2 Guardar tus cambios](#)
 - [3.2.3 Subir los cambios a la rama dev](#)
 - [3.3 Corrección de commits \(LOCAL\)](#)
 - [3.3.1 Quitar un archivo del último commit](#)
 - [3.3.2 Agregar un archivo del último commit](#)
 - [3.3.3 Cambiar el título o la descripción](#)
 - [3.3.4 Deshacer un commit entero y rehacerlo](#)
- [4. Convención de nombres y código en Unity](#)
 - [4.1 Reglas generales](#)
 - [4.1.1 USO DE ESPAÑOL](#)
 - [4.2 Nomenclatura de Variables](#)
 - [4.3 Ejemplo de código](#)

Introducción

Este documento servirá como guía de referencia para el desarrollo del proyecto de Unity. Con este documento se intentará evitar las malas prácticas del desarrollo en conjunto para disminuir las posibilidades de corrupción de archivos y desincronización al trabajar.

1. Repositorio y Proyecto

1.1 Clonación del repositorio

Se intuye que el usuario ya tiene instalado tanto **Git** cómo **UnityHub** en su equipo

Abre git bash en la carpeta donde quieras que esté el proyecto de Unity Clona el repositorio, esto te **creará una nueva carpeta**:

```
git clone https://github.com/fabricio-cesur/WeaponMaster.git
```

Entra a la carpeta y revisa que te aparece tener la rama **(main)**:

```
cd WeaponMaster
```

1.2 Inicialización del proyecto en Unity

IMPORTANTE: Instalar la versión de Unity **6000.0.50f1**, esto debido a que no se puede actualizar a una versión más reciente por la falta de permisos en los ordenadores del centro. Ignorar advertencias de seguridad.

- Abre **UnityHub** y en proyectos dale a la opción → **Add** → **Add project from disk**
- Selecciona la carpeta donde tengas guardado el repositorio y haz click en **Open**

Esto creará todos los archivos locales necesarios para trabajar en el proyecto.

1.3 Configuración de Unity

Es **imprescindible** configurar Unity para que fuerce los archivos binarios a ser YAML (texto legible para Git). Esto se hace de la siguiente manera:

1. Abrir el proyecto
2. Al tenerlo abierto ve a **Edit** → **Project Settings**
3. Entra a la sección de **Editor**
4. Busca la sección **Asset Serialization**
5. Asegúrate que el **Mode** está en **Force Text**.

De estar en modo "Binary" o "Mixed" escribirá las escenas y paquetes en formato binario, lo que creará conflictos irreparables. Al tenerlo en "Force Text", Git tiene la posibilidad de fusionar cambios en escenas.

2. Prefijos

2.1 Acciones

Lo que se hizo en el commit, explican **QUÉ** se hizo.

Prefijo	Uso recomendado
FEAT	Nueva funcionalidad: Código nuevo para mecánicas o lógica del juego
FIX	Corrección de errores: Bugs, errores de compilación o fallos lógicos
ADD	Agregar recursos: Assets, sprites, escenas, prefabs, sonidos, música
CHANGE	Modificaciones: Nombre de variables, movimiento de archivos, lógica
DELETE	Eliminación: Archivos, assets, scripts o escenas que ya son obsoletos
REFACTOR	Refactorización: Limpieza de código sin afectar funcionamiento
IMPORT	Importación externa: Recursos de terceros que no son propios

Prefijo	Uso recomendado
WIP	En progreso: Trabajo que aún no está completo ni funcionana
BALANCE	Ajustes: Daño, velocidad, timings, comportamiento de las mecánicas
AUTO	Cambios automáticos: No fueron hechos por ti sino por el programa en sí
INIT	Creación: Creación de archivos que tienen poco o nulo contenido
SETUP	Preparación: Establecer bases para algo futuro, crear carpetas, settings
MERGE	Juntar: Hacer merge de los cambios a dev a main u otras ramas
BUILD	Builds: Preparación para hacer builds, configuración de plataformas
RELEASE	Ejecutable: Se crea o sube el archivo ejecutable de una versión ya lista

2.2 Áreas

A qué afectó el commit, explican el **DÓNDE** se hizo

Prefijo	Uso recomendado
doc	Documentación, README, guías, comentarios
art	Sprites, música, partículas, texturas, materiales
prefab	Objetos prefabricados de Unity
scene	Escenas (.unity): objetos, layout, jerarquía
input	Controles, mapeo de teclas, InputSystem
anim	Animaciones, AnimationControllers, transición entre animaciones
tag	TagManager, crear un tag, agregar o cambiar los tags de un objeto
ui	Canvas, Menu, botones, HUD, interfaz, paneles
audio	Sonido, música, efectos, configuración de audio
ai	Comportamiento de enemigos, npcs, pathfinding
phys	Físicas del juego u objetos, colliders, rigidbodies, triggers
config	Configuración o preferencias del proyecto o del editor

2.3 Uso de Prefijos

Al utilizar los prefijos en los commits es necesario ser constantes para mantener la legibilidad y el orden del proyecto.

Ejemplo:

```
git commit -m "<PREFIJO DE ACCIÓN> (<prefijo de área>): <Título>" -m "<Descripción>"
```

```
git commit -m "FEAT(input): Movimiento básico de personaje" -m "Se creó el script que manejará los inputs del personaje para su movimiento de lado a lado y el salto."
```

- Los prefijos siempre se escribirán en **inglés**.
- Los prefijos de **acción** se escribirán en **mayúsculas**.
- Los prefijos de **área** se escribirán en **minúsculas**.
- Los prefijos de **área** no son obligatorios a usar en los commits.
- Los prefijos de **área** no deben ser sólo los listados en [la lista](#), también se pueden crear nuevos, estos eran más por dar ejemplos.
- La descripción es opcional, sobretodo para explicar lo que no quepa en el título.

3. Flujo de trabajo

Se trabajará en una rama diferente para cada integrante, teniendo otra rama de desarrollo y la main que será la rama más limpia con la versión segura y libre de fallos conocidos.

rama propia --> dev --> main

- Rama **propia**: La más segura para hacer cambios personales y trabajar.
- Rama **dev**: Aquí se juntan los cambios personales armándose una versión limpia del trabajo.
- Rama **main**: Las versiones finales o releases, completamente limpia y revisadas.

3.1 Configuración inicial

Configuración **inicial** teniendo en cuenta implícitamente que la rama **dev** ya fue creada en base de la rama main. Esto debe hacerse *solo una vez* para preparar el entorno de trabajo de git.

- Se asume que el git ya está configurado con tu cuenta de git
- Tu rama propia debería de ser tu nombre

```
git checkout dev          # Asegurarse hacer la rama desde la dev
git push -u origin dev    # El push por defecto será de la dev local a
la dev remota
git checkout -b [tu-rama]  # Crear tu rama para trabajar
git push -u origin [tu-rama] # El push por defecto será de la local a la
remota
git config pull.rebase true # Utilizar rebase en el pull envés de merge
```

3.2 flujo común al hacer cambios

3.2.1 Antes de cualquier cambio

A diario antes de trabajar y hacer commits, sincronizar con la rama dev para estar lo más sincronizados posible. Hacer esto antes de cualquier cambio o empezar a trabajar para así estar igual que la rama dev.

```
git checkout <tu-rama>           # Asegurarse estar en tu rama propia
git pull origin <tu-rama>         # Sincronizarte con tu propia
rama remota
git pull origin dev               # Sincronizarte con los cambios de la dev
remota

# Hacer cambios en Unity o el proyecto
```

3.2.2 Guardar tus cambios

Después de tener algunos cambios hechos con cambios medianamente estables o bien dirigidos, se guardará en un commit local con un mensaje claro y descripción si es necesario.

```
# Estando en tu rama propia.
git add .                      # Añadir todos los cambios
git commit -m "PREFIJO: Mensaje" # Escribir mensaje con prefijo
git push origin <tu-rama>      # Subir cambios al remoto
```

3.2.3 Subir los cambios a la rama dev

Después de tener algunos commits hechos con alguna feature o añadiendo nuevas cosas, se subirán al remoto de la rama propia para luego mergearla a la rama dev.

```
# Estando en tu (rama propia)
git pull origin dev           # Estar sincronizados con la dev
# +---Puede haber conflictos, resolverlos y seguir como se indica en el bloque de
abajo---+
git push origin <tu-rama>     # Subir la resolución de conflictos
git checkout dev              # Cambiar a la rama dev
# Estando en la rama (dev)
git pull origin dev           # Traer la dev actualizada del repositorio
git merge <tu-rama>           # Hacer un merge de los nuevos commits en la
rama dev
git push origin dev           # Subir el nuevo merge a la dev remota
# NO te olvides VOLVER a tu rama
git checkout <tu-rama>        # Regresar a tu rama propia
```

En caso de que haya habido **conflictos** en Unity con el `git pull origin dev`, se deben resolver dichos conflictos y luego seguir los comandos siguientes:

```
# Después de haber resuelto los conflictos
git add .                      # Añadir los archivos con conflictos resueltos
git commit -m "MERGE: Conflicto x resuelto"
# +---Continuar con el flujo normal con checkout merge como se indica en el bloque
de arriba---+
```

3.3 Corrección de commits (LOCAL)

Puede pasar que **el último commit** que hiciste tuvo un archivo innecesario o que era para otro commit, o que simplemente no escribiste bien el título o la descripción. Para estos casos hay varios comandos que pueden ayudar.

IMPORTANTE: todos estos cambios se tienen que hacer en **local** y **antes** de hacer el **push**. Si el commit erróneo ya se subió a la rama remota (*en GitHub*) se debe resolver con otro commit.

3.3.1 Quitar un archivo del último commit

```
git reset HEAD archivo.file      # Saca el archivo del último commit
git commit --amend --no-edit     # Reescribe el último commit sin el archivo
```

3.3.2 Agregar un archivo del último commit

```
git add archivo.file            # Añade el archivo para el amend
git commit --amend --no-edit    # Reescribe el último commit con el nuevo
archivo
```

3.3.3 Cambiar el título o la descripción

```
# Cambiar solo el título
# (Si tenía descripción SE BORRARÁ)
git commit --amend -m "PREFIJO: Nuevo título"
# Cambiar el título Y la descripción (escribe el mismo título para no cambiarlo)
git commit --amend -m "PREFIJO: Nuevo título" -m "Nueva descripción"
```

Si quieres cambiar sólo el título y mantener la descripción sin tener que escribirla de nuevo tendrás que escribir `git commit --amend` y se abrirá una consola **NANO** para configurar el texto del commit.

3.3.4 Deshacer un commit entero y rehacerlo

```
# deshace los cambios del último commit sin deshacer los cambios de este
# Te mueve al commit ANTERIOR dejando los archivos listos en staged
git reset --soft HEAD~1
# Hace un commit completamente nuevo cómo si el último no hubiera pasado
git commit -m "PREFIJO: Título del commit"
```

4. Convención de nombres y código en Unity

Se seguirá el estándar de C# y Unity, así se mantendrá el código limpio y homogéneo con respecto a los métodos y variables nativas de Unity.

4.1 Reglas generales

1. Las **carpetas** y **archivos** se escribirán en **inglés**.
2. Las **variables** y **métodos** se escribirán en **español**.
3. Todos los Scripts en la carpeta **Assets/Scripts/**.
4. Seguir el siguiente orden en los Scripts:
 1. Importes
 2. Declaración de Clase
 3. Variables Constantes
 4. Variables Inspector
 5. Variables Públicas
 6. Variables Privadas
 7. Métodos Unity
 8. Métodos propios

4.1.1 USO DE ESPAÑOL

- Se usará únicamente en variables y métodos
- **NO** se usará la "Ñ", se remplazará con "N".
- **NO** se usarán tildes (á, é, í, ó, ú)
- **NO** se usarán símbolos de apertura. (¿, ¡)

4.2 Nomenclatura de Variables

Elemento	Case	Notas
Carpetas	PascalCase	Estructura clara y nombres en inglés
Prefabs	PascalCase	Primera palabra su grupo (EnemyOrc, EnemyZombie)
Escenas	PascalCase	
Scripts	PascalCase	Su clase debe tener el mismo nombre
Clases	PascalCase	Deben tener el mismo nombre que el archivo
Métodos	PascalCase	Los métodos nativos de unity son iguales (Start)
Variables privadas	_camelCase	Se le pone un _ al inicio para diferenciarlo
Variables públicas	camelCase	Variables accesibles desde otros scripts/objetos
Variables Booleanas	camelCase	Utilizar prefijos es, tiene, puede cuando se pueda
Constantes	SCREAMING_SNAKE	Valores constantes y estáticos
Imágenes / Sprites	snake_case	A veces facilita la exportación en herramientas de arte

4.3 Ejemplo de código

```
using UnityEngine;

public class PlayerCombat : MonoBehaviour
{
    private const int LIMITE_GOLPES = 3;

    [Header("Configuración")]
    [SerializeField] private float rangoAtaque = 1.5f;

    private int _vidaActual;
    public bool puedeAtacar;

    private void Start()
    {
        _vidaActual = 100;
    }

    public void RecibirDano(int cantidadDano)
    {
        float danoFinal = cantidadDano * 1.5f;
        _vidaActual -= (int)danoFinal;
    }
}
```

Esta guía sirve como referencia profesional y asegura que el historial de Git sea limpio, legible y coherente con buenas prácticas de desarrollo en Unity 2D. **Última actualización: 28/01/2026**